

PowerShell Basics

Adam Driscoll



Adam Driscoll

Twitter: @adamdriscoll

adriscoll@concurrency.com

Senior Application Developer



Agenda

- What is PowerShell?
- Tooling
- Language Basics
- Pipeline
- Help System
- Module Basics
- Lab



What is PowerShell?

- Scripting language for Windows
- Available since Windows Server 2008
- Object based pipeline
- Built-on .NET
- Currently at version 5



Tooling

- PowerShell.exe
- PowerShell ISE
- VS Code with PowerShell Extension
- Visual Studio with PowerShell Extension
- PowerShell Studio
- ISE Steroids



Language Basics

- Commands
- Syntax
- Help System
- Pipeline
- Drive System
- Modules



Commands

- Verb-Noun Syntax
- Discoverable
- Parameters



Command Format

Get-Process

Standard Verb

Singular Noun

Get-SCSMObject

Frequently prefixed



```
PS C:\Users\adriscoll> Get-Verb
```

Verb	Group
----	-----
Add	Common
Clear	Common
Close	Common
Copy	Common
Enter	Common
Exit	Common
Find	Common
Format	Common
Get	Common
Hide	Common
Join	Common
Lock	Common
Move	Common
New	Common
Open	Common
Optimize	Common
Pop	Common
Push	Common
Redo	Common
Remove	Common
Rename	Common
Reset	Common
Resize	Common
Search	Common
Select	Common
Set	Common
Show	Common
Skip	Common
Split	Common
Step	Common
Switch	Common
Undo	Common
Unlock	Common
Watch	Common

98 Standard Verbs

Command Parameters

```
1 Get-Process -Name PowerShell -IncludeUserName|
```

```
PS C:\Windows\system32> Get-Process -Name PowerShell -IncludeUserName
```

Handles	WS(K)	VM(M)	CPU(s)	Id	UserName	ProcessName
-----	-----	-----	-----	--	-----	-----
654	78240	...95		14820	CONCURRENCY\ad...	powershell



Get-Help about_Parameters



Common Parameters

- -Debug (db)
- -ErrorAction (ea)
- -ErrorVariable (ev)
- -InformationAction
- -InformationVariable
- -OutVariable (ov)
- -OutBuffer (ob)
- -PipelineVariable (pv)
- -Verbose (vb)
- -WarningAction (wa)
- -WarningVariable (wv)



Get-Help about_CommonParameters



Parameter Sets

- Defines grouped sets of parameters
- Changes syntax of command based on parameters specified

Parameter Set 1

SYNTAX

```
Start-Process [-FilePath] <string> [[-ArgumentList] <string[]>] [-Credential <pscredential>] [-WorkingDirectory <string>] [-LoadUserProfile] [-NoNewWindow] [-PassThru] [-RedirectStandardError <string>] [-RedirectStandardInput <string>] [-RedirectStandardOutput <string>] [-WindowStyle <ProcessWindowStyle> {Normal | Hidden | Minimized | Maximized}] [-Wait] [-UseNewEnvironment] [<CommonParameters>]

Start-Process [-FilePath] <string> [[-ArgumentList] <string[]>] [-WorkingDirectory <string>] [-PassThru] [-Verb <string>] [-WindowStyle <ProcessWindowStyle> {Normal | Hidden | Minimized | Maximized}] [-Wait] [<CommonParameters>]
```

Parameter Set 2



Command Discovery

1 `Get-Command -Noun Process*`

```
PS C:\Users\adriscoll> Get-Command -Noun Process*
```

CommandType	Name
-----	----
Cmdlet	Debug-Process
Cmdlet	Get-Process
Cmdlet	Start-Process
Cmdlet	Stop-Process
Cmdlet	Wait-Process



Help System

- Get-Help
- Command Based Help
- About Files
- Extensible
- Updatable



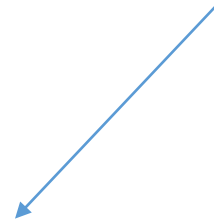
Get-Help Get-Help



Variables

```
1 $Name = 'Adam Driscoll'  
2 $Age = 29  
3 $Birthday = [DateTime] '5\6\1986'
```

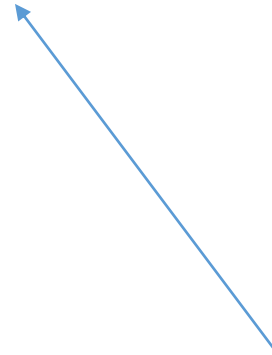
Value



Variable Name



Data Type



Get-Help about_Variables

Strings

```
1  $Name = 'Literal string'
2
3  "Expandable string $Name"
4
5  @'
6  | Here string will preserve
7  | new line characters
8  | '@
```

Conditional Operators

Condition

```
1  if ($IsComputer)
2  {
3      "Computer"
4  }
5  elseif ($IsUser)
6  {
7      "User"
8  }
9  else
10 {
11     "Unknown"
12 }
```

Expression Body



Get-Help about_If

Comparison Operators

```
1 'Adam' -eq 'Bob'      #False
2 'Adam' -ne 'Bob'      #True
3 1 -lt 10              #True
4 1 -gt 11              #False
5 'Adam' -match 'Ad'    #True
6 'Adam' -like 'Ad*'    #True
7 @('Adam', 'Bob') -contains 'Bob' #True
```



Get-Help about_Operators



Arrays

```
1 $Names = @('Adam', 'Bob', 'Bill')  
2 $Names += 'John'  
3 $Names.Length |
```



Get-Help about_Arrays



Loops

Get-Help about_ForEach
Get-Help about_For
Get-Help about_While

```
1 foreach($Process in $Processes)
2 {
3     $Process.Id
4 }
```

```
1 $i = 0
2 while($i -lt 10)
3 {
4     $i++
5 }
```

```
1 for($i = 0; $i -lt 10; $i++)
2 {
3     $i
4 }
```



Objects

- Objects rather than text
- Objects have properties and methods
- Objects can be formatted, filtered and extended



Get-Help about_Objects



Get-Member

```
PS C:\Users\adriscoll> [DateTime]::Now | Get-Member
```

TypeName: System.DateTime

Name	MemberType	Definition
-----	-----	-----
Add	Method	datetime Add(timespan value)
AddDays	Method	datetime AddDays(double value)
AddHours	Method	datetime AddHours(double value)
AddMilliseconds	Method	datetime AddMilliseconds(double value)
AddMinutes	Method	datetime AddMinutes(double value)
AddMonths	Method	datetime AddMonths(int months)
AddSeconds	Method	datetime AddSeconds(double value)
AddTicks	Method	datetime AddTicks(long value)
AddYears	Method	datetime AddYears(int value)
CompareTo	Method	int CompareTo(System.Object value) int Com



Pipeline

- Chain commands together to perform aggregate operations
- Simply looping statements
- Deal with sets of objects easier



Get-Help about_Pipelines



Aliases

- Shorter syntax for commands
- Can be exported from modules
- Can be defined on the fly
- Many built in aliases



Get-Help about_Aliases



Module Basics

- Modules are collection of commands, formatting files, aliases or providers
- Can be loaded on demand
- Module installation locations (\$Env:PSModulePath)
- Install-Module – PowerShell Gallery



Get-Help about_Modules



Lab

1. Get a list of processes that are using more than 100mb of RAM
2. Display all running services
3. Update the help on your machine. Find and use the command to display command history in your prompt.
4. List all modules currently installed on your machine.
5. Install a new module from either the PowerShell Gallery or GitHub
6. List all commands available in the module

